

Master Memory Leaks in React.js

A **memory leak** happens when memory is allocated but never released — objects that should be garbage-collected stay alive because **something still references them**.

In React, leaks happen when components don't clean up resources properly before unmounting — like:

- Async calls still resolving
- Subscriptions that never unsubscribe
- Event listeners never removed
- Timers that keep ticking
- Closures keeping stale references

Common Causes of Memory Leaks in React:

1. Event Listeners:

If you add event listeners to the DOM, window, or document and don't remove them when the component unmounts, memory can get stuck.

2. setState on Unmounted Components:

Calling setState after a component is removed from the screen keeps old data in memory.

3. Subscriptions and External Resources:

Not unsubscribing from things like WebSockets, APIs, or data streams can stop memory from being cleared.

4. Closures:

Functions that “remember” old state or props can accidentally hold memory, even after the component is gone.

Debug a Memory Leak

Use Chrome DevTools → Performance Tab

- Open your app in Chrome.
- Go to **Performance** → **Record**.
- Interact with your app (navigate, mount/unmount components).
- Stop recording and **look for memory spikes** that never drop.
 - If memory keeps growing, that's a leak.

Use Chrome DevTools → Memory Tab

- Go to **Memory** tab → select **Heap Snapshot**.
- Take a snapshot → interact with the app → take another snapshot.
- Compare them.
- Look for:
 - Detached DOM nodes (meaning something references removed components)
 - React component instances not being garbage-collected.

Use React Developer Tools Profiler

- Record interactions.
- Check which components re-render unexpectedly or stay mounted longer than expected.
- Often reveals components keeping subscriptions, refs, or large data in memory.

